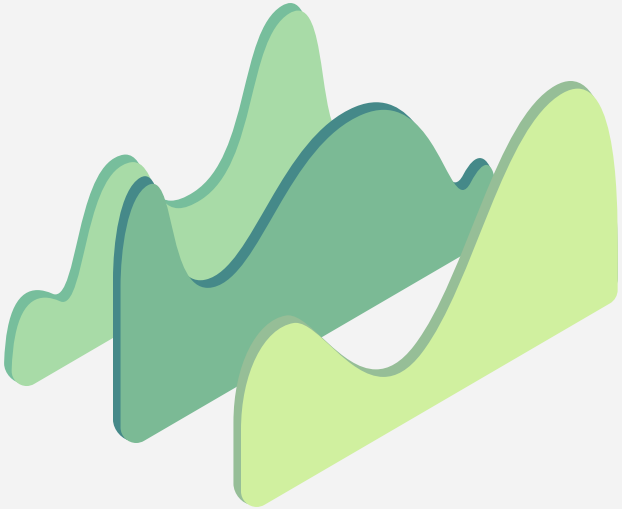
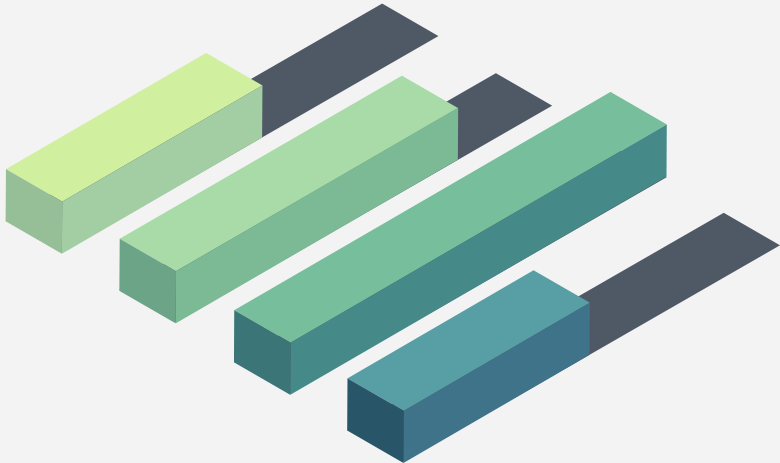
200Lab Education

SELECT Statement

Van Anh - Data Mentor





Definitions

+ Definitions: Select is the command we use most often in SQL. It lets us select data (specified rows and columns) from one or more tables. Columns are selected by name, rows are selected with conditional statements (values of a particular column meeting some criteria).

+ Syntax:

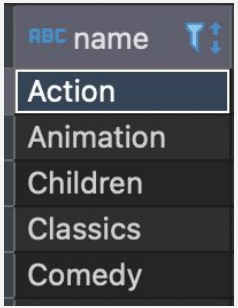
```
SELECT <columns>
FROM <table>;
```

```
SELECT *
FROM <table>;
```

Select Examples:

1. Select One Column:

```
SELECT
    name
FROM sakila.category;
```



name
Action
Animation
Children
Classics
Comedy

2. Select Multiple Columns:

```
SELECT
    category_id,
    name,
    last_update
FROM sakila.category;
```

```
SELECT
    *
FROM sakila.category;
```

category_id	name	last_update
1	Action	2006-02-15 04:46:27
2	Animation	2006-02-15 04:46:27
3	Children	2006-02-15 04:46:27
4	Classics	2006-02-15 04:46:27
5	Comedy	2006-02-15 04:46:27
6	Documentary	2006-02-15 04:46:27
7	Drama	2006-02-15 04:46:27
8	Family	2006-02-15 04:46:27
9	Foreign	2006-02-15 04:46:27
10	Games	2006-02-15 04:46:27
11	Horror	2006-02-15 04:46:27
12	Music	2006-02-15 04:46:27
13	New	2006-02-15 04:46:27
14	Sci-Fi	2006-02-15 04:46:27
15	Sports	2006-02-15 04:46:27
16	Travel	2006-02-15 04:46:27

How To Reorder The Table By Selecting

+ The ***ORDER BY*** keyword is used to sort the result-set in ascending or descending order.

+ The ***ORDER BY*** keyword sorts the records in ascending order by default. To sort the records in descending order, use the ***DESC*** keyword

+ Syntax:

```
SELECT column1, column2, ...
FROM table_name
ORDER BY column1, column2, ... ASC|DESC;
```

Order By Example

```
SELECT
  film_id,
  title
FROM film
ORDER BY film_id;
```



	film_id	title
1	1	ACADEMY DINOSAUR
2	2	ACE GOLDFINGER
3	3	ADAPTATION HOLES
4	4	AFFAIR PREJUDICE
5	5	AFRICAN EGG
6	6	AGENT TRUMAN
7	7	AIRPLANE SIERRA
8	8	AIRPORT POLLOCK

```
SELECT
  film_id,
  title
FROM film
ORDER BY film_id DESC;
```



	film_id	title
1	1,000	ZORRO ARK
2	999	ZOOLANDER FICTION
3	998	ZHIVAGO CORE
4	997	YOUTH KICK
5	996	YOUNG LANGUAGE
6	995	YENTL IDAHO
7	994	WYOMING STORM
8	993	WRONG BEHAVIOR

Order By Example (Cont.)

```
SELECT
    rental_id,
    rental_date,
    inventory_id,
    customer_id,
    return_date,
    staff_id,
    last_update
FROM sakila.rental
ORDER BY customer_id DESC, rental_id ASC, rental_date ASC;
```

```
SELECT
    rental_id,
    rental_date,
    inventory_id,
    customer_id,
    return_date,
    staff_id,
    last_update
FROM sakila.rental
ORDER BY 4 DESC, 1 ASC, 5 ASC;
```

	rental_id	rental_date	inventory_id	customer_id	return_date	staff_id	last_update
18	15,719	2005-08-23 11:08:46	3,990	599	2005-08-25 07:25:46	1	2006-02-15 21:30:53
19	15,725	2005-08-23 11:25:00	378	599	2005-08-26 11:46:00	1	2006-02-15 21:30:53
20	3,005	2005-06-20 10:10:29	286	598	2005-06-28 15:48:29	2	2006-02-15 21:30:53
21	3,648	2005-07-06 07:30:41	2,810	598	2005-07-10 06:00:41	2	2006-02-15 21:30:53
22	3,950	2005-07-06 21:48:44	4,153	598	2005-07-14 02:25:44	1	2006-02-15 21:30:53
23	3,972	2005-07-06 22:53:57	1,895	598	2005-07-11 01:32:57	1	2006-02-15 21:30:53
24	4,181	2005-07-07 10:27:54	3,725	598	2005-07-13 06:09:54	1	2006-02-15 21:30:53
25	5,688	2005-07-10 09:16:08	3,036	598	2005-07-15 09:44:08	1	2006-02-15 21:30:53
26	6,519	2005-07-12 04:00:36	4,149	598	2005-07-19 01:15:36	1	2006-02-15 21:30:53
27	6,528	2005-07-12 04:29:44	3,824	598	2005-07-18 02:39:44	2	2006-02-15 21:30:53
28	6,575	2005-07-12 06:12:53	1,414	598	2005-07-18 07:55:53	2	2006-02-15 21:30:53
29	6,660	2005-07-12 11:20:12	446	598	2005-07-20 12:58:12	2	2006-02-15 21:30:53
30	7,201	2005-07-27 08:57:40	482	598	2005-08-04 09:55:40	2	2006-02-15 21:30:53
31	7,354	2005-07-27 14:42:11	2,395	598	2005-08-03 18:19:11	2	2006-02-15 21:30:53
32	7,998	2005-07-28 15:08:48	1,397	598	2005-07-31 16:14:48	2	2006-02-15 21:30:53
33	8,436	2005-07-29 07:21:20	2,702	598	2005-07-31 12:56:20	2	2006-02-15 21:30:53
34	8,511	2005-07-29 09:42:42	3,667	598	2005-08-06 14:22:42	2	2006-02-15 21:30:53
35	8,939	2005-07-30 02:56:53	4,175	598	2005-08-01 21:19:53	1	2006-02-15 21:30:53
36	10,054	2005-07-31 19:15:52	2,303	598	2005-08-04 19:54:52	1	2006-02-15 21:30:53
37	11,350	2005-08-02 17:22:59	1,265	598	2005-08-09 19:56:59	2	2006-02-15 21:30:53
38	12,601	2005-08-18 16:47:52	3,099	598	2005-08-24 11:05:52	1	2006-02-15 21:30:53
39	14,345	2005-08-21 08:41:15	1,726	598	2005-08-24 11:59:15	1	2006-02-15 21:30:53
40	15,307	2005-08-22 19:54:26	4,525	598	2005-08-29 01:38:26	2	2006-02-15 21:30:53
41	15,443	2005-08-23 00:44:15	742	598	2005-09-01 05:33:15	2	2006-02-15 21:30:53
42	34	2005-05-25 04:19:28	2,613	597	2005-05-29 00:10:28	2	2006-02-15 21:30:53
43	514	2005-05-28 03:09:28	1,697	597	2005-06-05 00:49:28	2	2006-02-15 21:30:53

Select Distinct

+ **Definitions:** ***DISTINCT*** is used in a ***SELECT*** clause to return unique combinations of data across all the columns returned by the query.

+ It comes before the list of column names, and applies to the combination of all columns.

+ **Syntax:**

```
SELECT DISTINCT column1, column2, ...
FROM table_name;
```

Example: Select Distinct

```
SELECT
  DISTINCT
  actor_id
FROM sakila.film_actor;
```



	actor_id
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8

```
SELECT
  DISTINCT
  customer_id,
  staff_id
FROM payment;
```

	customer_id	staff_id
1	1	1
2	1	2
3	2	1
4	2	2
5	3	1
6	3	2
7	4	1
8	4	2

Use Limit

+ Definitions: The ***LIMIT*** clause is used to specify the number of records to return.

+ The ***LIMIT*** clause is useful on large tables with thousands of records. Returning a large number of records can impact performance.

+ Syntax:

```
SELECT
    column_name(s)
FROM table_name
LIMIT number;
```

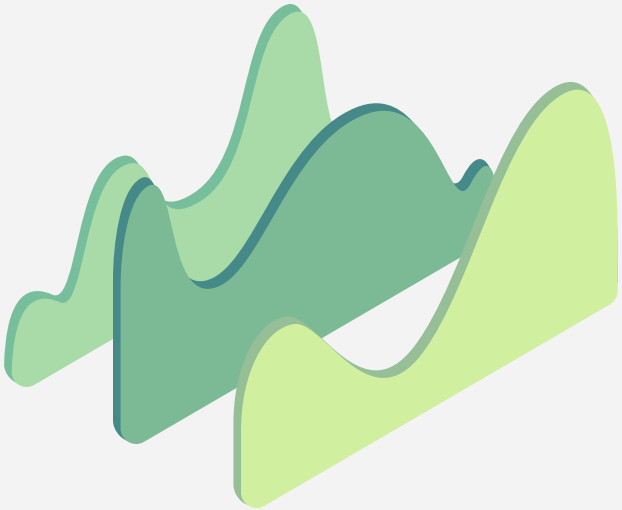
Limit Example

```
SELECT
  customer_id,
  store_id,
  first_name,
  last_name,
  email
FROM sakila.customer
LIMIT 5;
```

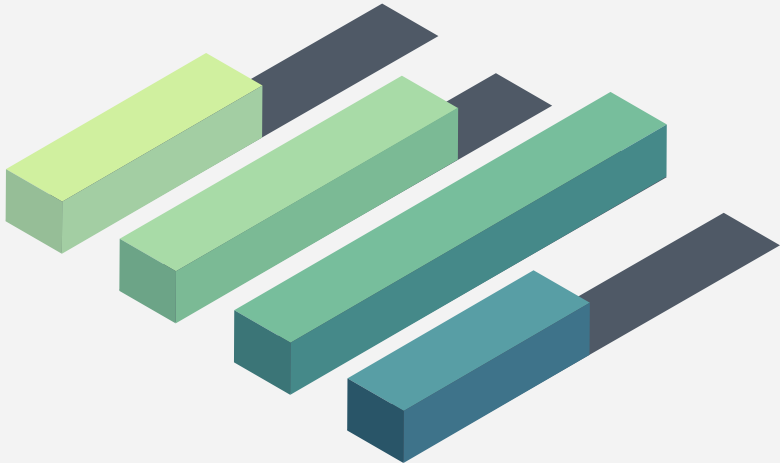
customer_id	store_id	first_name	last_name	email
1	1	MARY	SMITH	MARY.SMITH@sakilacustomer.org
2	1	PATRICIA	JOHNSON	PATRICIA.JOHNSON@sakilacustomer.org
3	1	LINDA	WILLIAMS	LINDA.WILLIAMS@sakilacustomer.org
4	2	BARBARA	JONES	BARBARA.JONES@sakilacustomer.org
5	1	ELIZABETH	BROWN	ELIZABETH.BROWN@sakilacustomer.org

200Lab Education

Column Alias



Van Anh - Data Mentor



Definitions

+ Columns aliases are used to give a column in a table, a temporary name.
+ Aliases are often used to make column names more readable.
+ An alias only exists for the duration of that query.
+ An alias is created with the AS keyword

+ Syntax:

```
SELECT column_name AS alias_name
FROM table_name;
```

Some Example

```
SELECT
    category_id,
    name AS category_name,
    last_update
FROM sakila.category;
```

	category_id	category_name	last_update
1	1	Action	2006-02-15 04:46:27
2	2	Animation	2006-02-15 04:46:27
3	3	Children	2006-02-15 04:46:27
4	4	Classics	2006-02-15 04:46:27
5	5	Comedy	2006-02-15 04:46:27
6	6	Documentary	2006-02-15 04:46:27
7	7	Drama	2006-02-15 04:46:27
8	8	Family	2006-02-15 04:46:27
9	9	Foreign	2006-02-15 04:46:27
10	10	Games	2006-02-15 04:46:27
11	11	Horror	2006-02-15 04:46:27
12	12	Music	2006-02-15 04:46:27
13	13	New	2006-02-15 04:46:27
14	14	Sci-Fi	2006-02-15 04:46:27
15	15	Sports	2006-02-15 04:46:27
16	16	Travel	2006-02-15 04:46:27

	category_id	name	last_update
1	1	Action	2006-02-15 04:46:27
2	2	Animation	2006-02-15 04:46:27
3	3	Children	2006-02-15 04:46:27
4	4	Classics	2006-02-15 04:46:27
5	5	Comedy	2006-02-15 04:46:27
6	6	Documentary	2006-02-15 04:46:27
7	7	Drama	2006-02-15 04:46:27
8	8	Family	2006-02-15 04:46:27
9	9	Foreign	2006-02-15 04:46:27
10	10	Games	2006-02-15 04:46:27
11	11	Horror	2006-02-15 04:46:27
12	12	Music	2006-02-15 04:46:27
13	13	New	2006-02-15 04:46:27
14	14	Sci-Fi	2006-02-15 04:46:27
15	15	Sports	2006-02-15 04:46:27
16	16	Travel	2006-02-15 04:46:27

Some Example (Cont.)

```sql
SELECT
    film_id,
    title AS film_name,
    release_year AS publish_year,
    language_id
FROM sakila.film;
```

	film_id	film_name	publish_year	language_id
1	1	ACADEMY DINOSAUR	2006	1
2	2	ACE GOLDFINGER	2006	1
3	3	ADAPTATION HOLES	2006	1
4	4	AFFAIR PREJUDICE	2006	1
5	5	AFRICAN EGG	2006	1
6	6	AGENT TRUMAN	2006	1
7	7	AIRPLANE SIERRA	2006	1
8	8	AIRPORT POLLOCK	2006	1
9	9	ALABAMA DEVIL	2006	1
10	10	ALADDIN CALENDAR	2006	1
11	11	ALAMO VIDEOTAPE	2006	1

Some Example (Cont.)

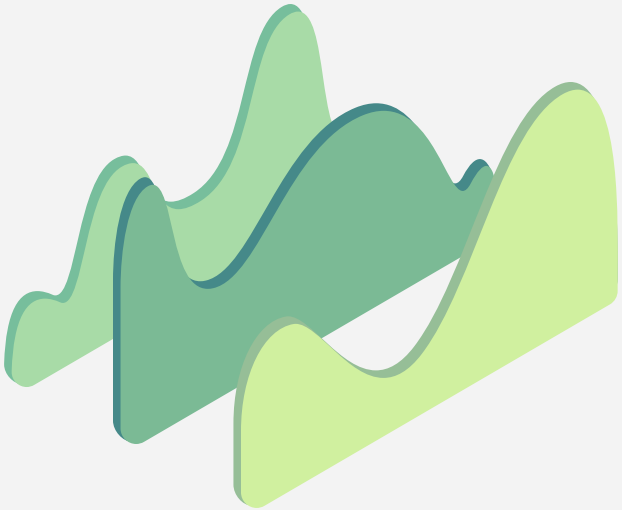
```sql
SELECT
    payment_id AS `Payment ID`,
    customer_id AS `Customer ID`,
    staff_id AS `Staff ID`,
    rental_id AS `Rental ID`,
    amount,
    payment_date AS `Payment Date`,
    last_update AS `Last Update`
FROM sakila.payment;
```

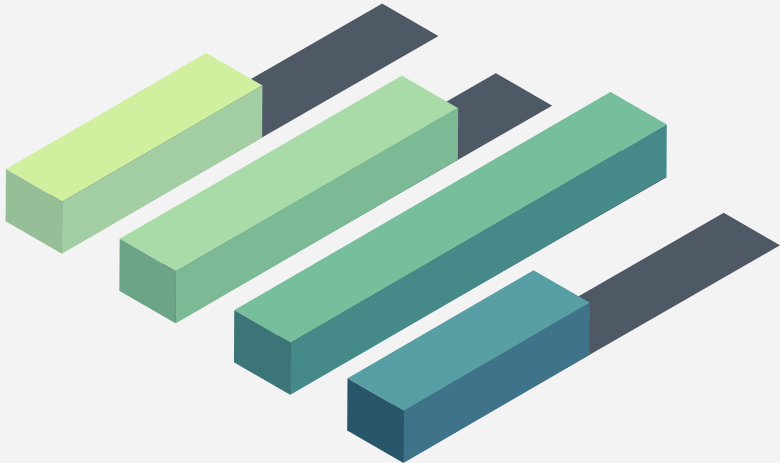
	Payment ID	Customer ID	Staff ID	Rental ID	amount	Payment Date	Last Update
1	1	1	1	76	2.99	2005-05-25 11:30:37	2006-02-15 22:12:30
2	2	1	1	573	0.99	2005-05-28 10:35:23	2006-02-15 22:12:30
3	3	1	1	1,185	5.99	2005-06-15 00:54:12	2006-02-15 22:12:30
4	4	1	2	1,422	0.99	2005-06-15 18:02:53	2006-02-15 22:12:30
5	5	1	2	1,476	9.99	2005-06-15 21:08:46	2006-02-15 22:12:30
6	6	1	1	1,725	4.99	2005-06-16 15:18:57	2006-02-15 22:12:30
7	7	1	1	2,308	4.99	2005-06-18 08:41:48	2006-02-15 22:12:30
8	8	1	2	2,363	0.99	2005-06-18 13:33:59	2006-02-15 22:12:30
9	9	1	1	3,284	3.99	2005-06-21 06:24:45	2006-02-15 22:12:30
10	10	1	2	4,526	5.99	2005-07-08 03:17:05	2006-02-15 22:12:30
11	11	1	1	4,611	5.99	2005-07-08 07:33:56	2006-02-15 22:12:30
12	12	1	1	5,244	4.99	2005-07-09 13:24:07	2006-02-15 22:12:30

200Lab Education

WHERE Clause

Van Anh - Data Mentor





Definition

+ **Definition:** The ***WHERE*** clause is used to filter records.

+ It is used to extract only those records that fulfill a specified condition.

+ **Syntax:**

```
SELECT
  column1,
  column2, ...
FROM table_name
WHERE condition;
```

Where Clause Example:

1. Example Where Clause Filter By A Numeric Value

```
SELECT
    *
FROM customer
WHERE store_id=2
LIMIT 10;
```

customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
4	2	BARBARA	JONES	BARBARA.JONES@sakilacustomer.c	8	1	2006-02-14 22:04:36	2006-02-15 04:57:20
6	2	JENNIFER	DAVIS	JENNIFER.DAVIS@sakilacustomer.o	10	1	2006-02-14 22:04:36	2006-02-15 04:57:20
8	2	SUSAN	WILSON	SUSAN.WILSON@sakilacustomer.or	12	1	2006-02-14 22:04:36	2006-02-15 04:57:20
9	2	MARGARET	MOORE	MARGARET.MOORE@sakilacustome	13	1	2006-02-14 22:04:36	2006-02-15 04:57:20
11	2	LISA	ANDERSON	LISA.ANDERSON@sakilacustomer.o	15	1	2006-02-14 22:04:36	2006-02-15 04:57:20
13	2	KAREN	JACKSON	KAREN.JACKSON@sakilacustomer.c	17	1	2006-02-14 22:04:36	2006-02-15 04:57:20
14	2	BETTY	WHITE	BETTY.WHITE@sakilacustomer.org	18	1	2006-02-14 22:04:36	2006-02-15 04:57:20
16	2	SANDRA	MARTIN	SANDRA.MARTIN@sakilacustomer.c	20	0	2006-02-14 22:04:36	2006-02-15 04:57:20
18	2	CAROL	GARCIA	CAROL.GARCIA@sakilacustomer.org	22	1	2006-02-14 22:04:36	2006-02-15 04:57:20
20	2	SHARON	ROBINSON	SHARON.ROBINSON@sakilacustom	24	1	2006-02-14 22:04:36	2006-02-15 04:57:20

Where Clause Examples (Cont.):

2. Example Where Clause Filter By A Text Value

```
SELECT
    *
FROM customer
WHERE first_name = 'BARBARA'
LIMIT 10;
```

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
1	4	2	BARBARA	JONES	BARBARA.JONES@sakilacustomer.c	8	1	2006-02-14 22:04:36	2006-02-15 04:57:20

Filtering Multiple Conditions Using AND

+ The ***WHERE*** clause can be combined with ***AND*** operators.

+ The ***AND*** operators are used to filter records based on more than one condition.

+ The ***AND*** operator displays a record if all the conditions separated by ***AND*** are ***TRUE***.

+ **Syntax:**

```
SELECT
  column1,
  column2, ...
FROM table_name
WHERE condition1
AND condition2
AND condition3 ...;
```

Filtering Multiple Conditions Examples

```sql
SELECT
  *
FROM customer
WHERE store_id=2
AND customer_id=400;
```

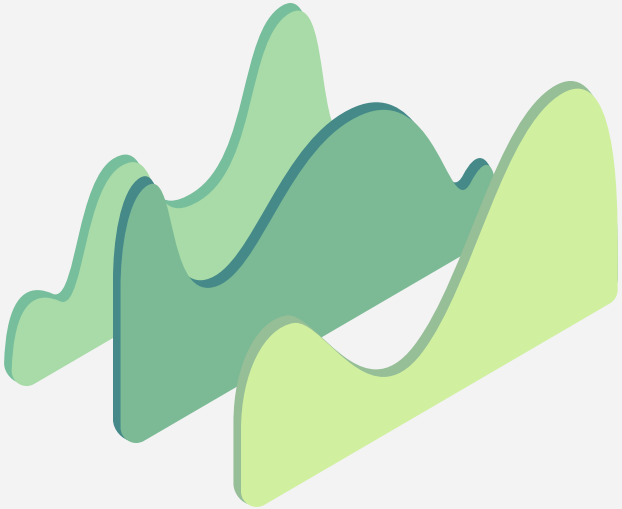
	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
1	400	2	BRYAN	HARDISON	BRYAN.HARDISON@sakilacustomer	405	1	2006-02-14 22:04:37	2006-02-15 04:57:20

```sql
SELECT
  *
FROM customer
WHERE store_id=2
AND first_name='Jon';
```

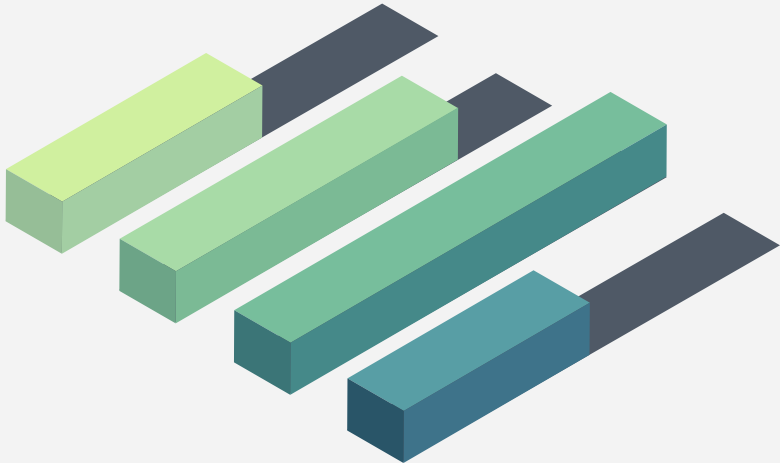
	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
1	455	2	JON	WILES	JON.WILES@sakilacustomer.org	460	1	2006-02-14 22:04:37	2006-02-15 04:57:20

200Lab Education

Operators in SQL



Van Anh - Data Mentor



Operator In SQL

SQL Arithmetic Operators

Operator	Description
+	Add
-	Subtract
*	Multiply
/	Divide
%	Modulo

Operator	Description
=	Equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
<>	Not equal to

SQL Comparison Operators

SQL Comparison Operators Example

```sql
-- Not Equal Operator
SELECT
    staff_id AS `Staff ID`,
    email AS `Email`,
    username AS `User Name`,
    last_update AS `Last Update`
FROM sakila.staff
WHERE username <> 'Jon';
```

	Staff ID	Email	User Name	Last Update
1	1	Mike.Hillyer@sakilastaff.com	Mike	2006-02-15 03:57:16

```sql
-- Greater than Operator
SELECT
  language_id,
  name,
  last_update
FROM sakila.`language`
WHERE language_id > 4;
```

	language_id	name	last_update
1	5	French	2006-02-15 05:02:19
2	6	German	2006-02-15 05:02:19

SQL Comparison Operators Example (Cont.)

```sql
-- Greater than or equal to
SELECT
    film_id,
    title,
    release_year,
    special_features,
    replacement_cost
FROM film WHERE film_id >= 995;
```

	film_id	title	release_year	special_features	replacement_cost
1	995	YENTL IDAHO	2006	Trailers,Commentaries,Deleted Scenes	11.99
2	996	YOUNG LANGUAGE	2006	Trailers,Behind the Scenes	9.99
3	997	YOUTH KICK	2006	Trailers,Behind the Scenes	14.99
4	998	ZHIVAGO CORE	2006	Deleted Scenes	10.99
5	999	ZOOLANDER FICTION	2006	Trailers,Deleted Scenes	28.99
6	1,000	ZORRO ARK	2006	Trailers,Commentaries,Behind the Scenes	18.99

SQL Comparison Operators Example (Cont.)

```sql
--  Less than Operator
SELECT
   language_id,
   name,
   last_update
FROM sakila.`language`
WHERE language_id < 4;
```

	language_id	name	last_update
1	1	English	2006-02-15 05:02:19
2	2	Italian	2006-02-15 05:02:19
3	3	Japanese	2006-02-15 05:02:19

```sql
-- Less Than Or Equal Operator
SELECT
  language_id,
  name,
  last_update
FROM sakila.`language`
WHERE language_id <=4;
```

	language_id	name	last_update
1	1	English	2006-02-15 05:02:19
2	2	Italian	2006-02-15 05:02:19
3	3	Japanese	2006-02-15 05:02:19
4	4	Mandarin	2006-02-15 05:02:19

Operator In SQL (Cont.)

Operator	Description
ALL	TRUE if all of the subquery values meet the condition
AND	TRUE if all the conditions separated by AND is TRUE
ANY	TRUE if any of the subquery values meet the condition
BETWEEN	TRUE if the operand is within the range of comparisons
EXISTS	TRUE if the subquery returns one or more records
IN	TRUE if the operand is equal to one of a list of expressions
LIKE	TRUE if the operand matches a pattern
NOT	Displays a record if the condition(s) is NOT TRUE
OR	TRUE if any of the conditions separated by OR is TRUE
SOME	TRUE if any of the subquery values meet the condition

SQL Logical Operators

And Operator

+ **Definition:** ***AND*** is the logical operators SQL uses to combine conditions in ***WHERE*** clauses in order to refine the data returned by a query.

+ ***TRUE*** if all the conditions separated by ***AND*** is ***TRUE***

+ **Example:**

```
-- And Operator
SELECT
    film_id,
    title,
    release_year
FROM film
WHERE film_id >= 995
AND release_year = 2006;
```

	film_id	title	release_year
1	995	YENTL IDAHO	2006
2	996	YOUNG LANGUAGE	2006
3	997	YOUTH KICK	2006
4	998	ZHIVAGO CORE	2006
5	999	ZOOLANDER FICTION	2006
6	1,000	ZORRO ARK	2006

Or Operator

+ **Definition:** ***Or*** is the logical operators SQL uses to combine conditions in ***WHERE*** clauses in order to refine the data returned by a query.

+ ***TRUE*** if any of the conditions separated by ***OR*** is ***TRUE***

+ **Example:**

```
-- OR Operator
SELECT
    rental_id,
    rental_date,
    inventory_id,
    customer_id,
    return_date,
    staff_id,
    last_update
FROM sakila.rental
WHERE customer_id = 399
OR customer_id  = 3
ORDER BY 1 DESC;
```

	rental_id	rental_date	inventory_id	customer_id	return_date	staff_id	last_update
1	15,619	2005-08-23 07:10:14	2,468	3	2005-08-26 07:21:14	2	2006-02-15 21:30:53
2	15,468	2005-08-23 01:25:30	2,440	399	2005-08-28 01:40:30	2	2006-02-15 21:30:53
3	15,300	2005-08-22 19:44:00	2,319	399	2005-08-25 22:49:00	2	2006-02-15 21:30:53
4	15,038	2005-08-22 09:37:27	1,685	3	2005-08-23 14:39:27	1	2006-02-15 21:30:53
5	14,699	2005-08-21 20:50:48	1,427	3	2005-08-29 18:08:48	2	2006-02-15 21:30:53
6	13,610	2005-08-20 06:14:12	2,526	3	2005-08-26 00:44:12	2	2006-02-15 21:30:53
7	13,403	2005-08-19 22:18:07	3,241	3	2005-08-27 19:23:07	1	2006-02-15 21:30:53
8	13,293	2005-08-19 18:35:52	380	399	2005-08-23 17:18:52	2	2006-02-15 21:30:53
9	13,253	2005-08-19 16:53:56	472	399	2005-08-20 11:38:56	2	2006-02-15 21:30:53
10	12,556	2005-08-18 14:49:55	1,984	3	2005-08-24 15:20:55	2	2006-02-15 21:30:53
11	11,953	2005-08-17 17:16:42	2,993	399	2005-08-23 18:28:42	1	2006-02-15 21:30:53
12	11,329	2005-08-02 16:42:52	2,677	399	2005-08-08 16:45:52	1	2006-02-15 21:30:53
13	10,960	2005-08-02 03:46:18	3,536	399	2005-08-11 01:29:18	1	2006-02-15 21:30:53
14	10,654	2005-08-01 16:31:35	2,522	399	2005-08-05 12:04:35	2	2006-02-15 21:30:53
15	10,597	2005-08-01 14:19:48	3,292	3	2005-08-08 20:01:48	1	2006-02-15 21:30:53
16	9,816	2005-07-31 11:32:58	4,560	3	2005-08-04 17:12:58	2	2006-02-15 21:30:53

In Operator

+ **Definition:** ***IN*** lets you specify a lot of values that you would otherwise join together with an ***OR*** statement.

+ ***TRUE*** if the operand is equal to one of a list of expressions

+ **Example:**

```
-- IN Operator
SELECT
    rental_id,
    rental_date,
    inventory_id,
    customer_id,
    return_date,
    staff_id,
    last_update
FROM sakila.rental
WHERE customer_id IN (399, 3)
ORDER BY rental_date DESC;
```

	rental_id	rental_date	inventory_id	customer_id	return_date	staff_id	last_update
1	15,619	2005-08-23 07:10:14	2,468	3	2005-08-26 07:21:14	2	2006-02-15 21:30:53
2	15,468	2005-08-23 01:25:30	2,440	399	2005-08-28 01:40:30	2	2006-02-15 21:30:53
3	15,300	2005-08-22 19:44:00	2,319	399	2005-08-25 22:49:00	2	2006-02-15 21:30:53
4	15,038	2005-08-22 09:37:27	1,685	3	2005-08-23 14:39:27	1	2006-02-15 21:30:53
5	14,699	2005-08-21 20:50:48	1,427	3	2005-08-29 18:08:48	2	2006-02-15 21:30:53
6	13,610	2005-08-20 06:14:12	2,526	3	2005-08-26 00:44:12	2	2006-02-15 21:30:53
7	13,403	2005-08-19 22:18:07	3,241	3	2005-08-27 19:23:07	1	2006-02-15 21:30:53
8	13,293	2005-08-19 18:35:52	380	399	2005-08-23 17:18:52	2	2006-02-15 21:30:53
9	13,253	2005-08-19 16:53:56	472	399	2005-08-20 11:38:56	2	2006-02-15 21:30:53
10	12,556	2005-08-18 14:49:55	1,984	3	2005-08-24 15:20:55	2	2006-02-15 21:30:53
11	11,953	2005-08-17 17:16:42	2,993	399	2005-08-23 18:28:42	1	2006-02-15 21:30:53
12	11,329	2005-08-02 16:42:52	2,677	399	2005-08-08 16:45:52	1	2006-02-15 21:30:53
13	10,960	2005-08-02 03:46:18	3,536	399	2005-08-11 01:29:18	1	2006-02-15 21:30:53
14	10,654	2005-08-01 16:31:35	2,522	399	2005-08-05 12:04:35	2	2006-02-15 21:30:53
15	10,597	2005-08-01 14:19:48	3,292	3	2005-08-08 20:01:48	1	2006-02-15 21:30:53

Like Operator

+ Definition: The ***LIKE*** operator is used in a ***WHERE*** clause to search for a specified pattern in a column.

+ There are two wildcards often used in conjunction with the ***LIKE*** operator:
  - The percent sign ***(%)*** represents zero, one, or multiple characters
  - The underscore sign ***(_)*** represents one, single character

+ Displays a record if the condition(s) is ***NOT TRUE***

+ Syntax:

```
SELECT
    column1,
    column2, ...
FROM table_name
WHERE columnN LIKE pattern;
```

Like Operator (Cont.)

+ Here are some examples showing different LIKE operators with '%' and '_' wildcards:

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that start with "a"
WHERE CustomerName LIKE '%a'	Finds any values that end with "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a_%'	Finds any values that start with "a" and are at least 2 characters in length
WHERE CustomerName LIKE 'a__%'	Finds any values that start with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that start with "a" and ends with "o"

Like Operator (Cont.)

+ Examples:

```
SELECT
    *
FROM sakila.customer
WHERE first_name LIKE 'MA%';
```

	customer_id	store_id	first_name
1	1	1	MARY
2	7	1	MARIA
3	9	2	MARGARET
4	38	1	MARTHA

```
SELECT
    *
FROM sakila.customer
WHERE first_name LIKE '%MA%';
```

	customer_id	store_id	first_name
10	128	1	MARJORIE
11	134	1	EMMA
12	177	2	SAMANTHA
13	178	2	MARION

```
SELECT
    *
FROM sakila.customer
WHERE first_name LIKE '%MA';
```

	customer_id	store_id	first_name
1	94	1	NORMA
2	122	1	THELMA
3	134	1	EMMA
4	196	1	ALMA

```
SELECT
    *
FROM sakila.customer
WHERE first_name LIKE '__MA%';
```

	customer_id	store_id	first_name
1	134	1	EMMA
2	177	2	SAMANTHA
3	196	1	ALMA
4	229	1	TAMARA

```
SELECT
    *
FROM sakila.customer
WHERE first_name LIKE 'm%n';
```

	customer_id	store_id	first_name
1	80	1	MARILYN
2	151	2	MEGAN
3	178	2	MARION
4	226	2	MAUREEN

Between Operator

+ The keyword ***BETWEEN*** returns a range of data between two values. It can be used with numbers, text strings and dates, and is most commonly found in the ***WHERE*** clause.

+ Example:

```
-- Between Operator
SELECT
    film_id,
    title,
    release_year
FROM film
WHERE film_id BETWEEN 1 AND 5;
```

	film_id	title	release_year
1	1	ACADEMY DINOSAUR	2006
2	2	ACE GOLDFINGER	2006
3	3	ADAPTATION HOLES	2006
4	4	AFFAIR PREJUDICE	2006
5	5	AFRICAN EGG	2006

Not Operator

+ Definition: The last logical operation used in the ***WHERE*** clause is ***NOT*** which is most commonly used with other keywords such as ***BETWEEN, LIKE*** or ***IN ...*** However it can also be used to negate an entire section in the WHERE clause.

+ Displays a record if the condition(s) is ***NOT TRUE***

+ Example:

```
SELECT
  column1,
  column2, ...
FROM table_name
WHERE columnN NOT condition;
```

Not Operator (Cont.)

+ Examples:

```
SELECT
    *
FROM sakila.customer
WHERE first_name  LIKE 'm%n'
AND first_name NOT LIKE '%lyn%';
```

	customer_id	store_id	first_name
1	151	2	MEGAN
2	178	2	MARION
3	226	2	MAUREEN
4	246	1	MARIAN

```
SELECT
    *
FROM sakila.customer
WHERE first_name  LIKE 'm%n'
AND first_name NOT IN ('MEGAN', 'MARION');
```

	customer_id	store_id	first_name
1	80	1	MARILYN
2	226	2	MAUREEN
3	246	1	MARIAN
4	383	1	MARTIN

```
SELECT
    *
FROM sakila.customer
WHERE first_name  LIKE 'm%n'
AND customer_id NOT BETWEEN 90 AND 550;
```

	customer_id	store_id	first_name
1	80	1	MARILYN
2	588	1	MARION

Combine Multiple Operator

```sql
SELECT
    customer_id,
    store_id,
    first_name,
    last_name,
    email,
    address_id,
    active,
    create_date,
    last_update
FROM sakila.customer
WHERE active = 1
AND  address_id BETWEEN 5 AND 10
AND (first_name LIKE 'm%n'
OR first_name NOT LIKE '_a%')
ORDER BY create_date DESC
LIMIT 10
```

customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
3	1	LINDA	WILLIAMS	LINDA.WILLIAMS@sakilacustomer.org	7	1	2006-02-14 22:04:36	2006-02-15 04:57:20
5	1	ELIZABETH	BROWN	ELIZABETH.BROWN@sakilacustomer.org	9	1	2006-02-14 22:04:36	2006-02-15 04:57:20
6	2	JENNIFER	DAVIS	JENNIFER.DAVIS@sakilacustomer.org	10	1	2006-02-14 22:04:36	2006-02-15 04:57:20

Combine Multiple Operator (Cont)

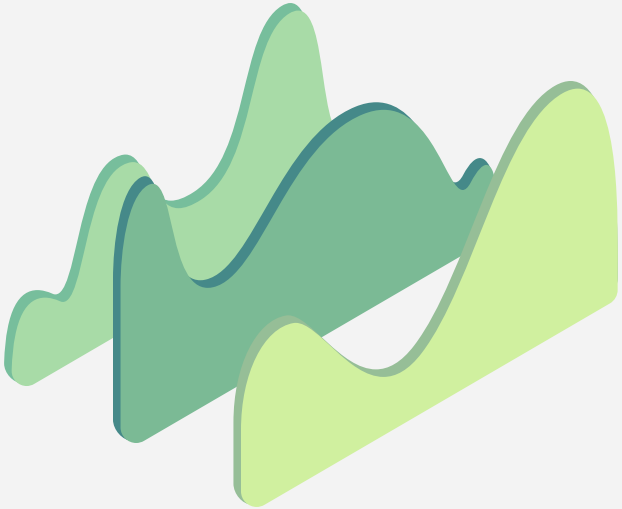
```sql
SELECT
    customer_id,
    store_id,
    first_name,
    last_name,
    email,
    address_id,
    active,
    create_date,
    last_update
FROM sakila.customer
WHERE active = 1
AND  address_id BETWEEN 5 AND 10
AND first_name LIKE 'm%n'
OR first_name NOT LIKE '_a%'
ORDER BY create_date DESC
LIMIT 10
```

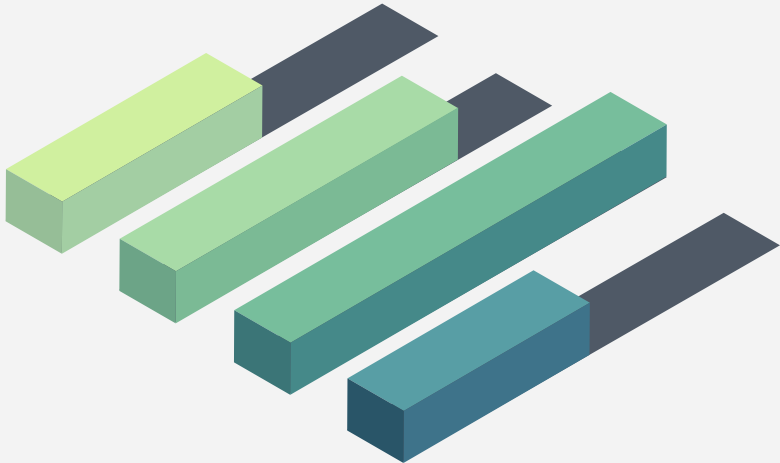
customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
278	2	BILLIE	HORTON	BILLIE.HORTON@sakilacustomer.org	283	1	2006-02-14 22:04:37	2006-02-15 04:57:20
284	1	SONIA	GREGORY	SONIA.GREGORY@sakilacustomer.org	289	1	2006-02-14 22:04:37	2006-02-15 04:57:20
281	2	LEONA	OBRIEN	LEONA.OBRIEN@sakilacustomer.org	286	1	2006-02-14 22:04:37	2006-02-15 04:57:20
279	2	DIANNE	SHELTON	DIANNE.SHELTON@sakilacustomer.org	284	1	2006-02-14 22:04:37	2006-02-15 04:57:20
599	2	AUSTIN	CINTRON	AUSTIN.CINTRON@sakilacustomer.org	605	1	2006-02-14 22:04:37	2006-02-15 04:57:20
283	1	FELICIA	SUTTON	FELICIA.SUTTON@sakilacustomer.org	288	1	2006-02-14 22:04:37	2006-02-15 04:57:20
277	2	OLGA	JIMENEZ	OLGA.JIMENEZ@sakilacustomer.org	282	1	2006-02-14 22:04:37	2006-02-15 04:57:20
282	2	JENNY	CASTRO	JENNY.CASTRO@sakilacustomer.org	287	1	2006-02-14 22:04:37	2006-02-15 04:57:20
280	2	TRACEY	BARRETT	TRACEY.BARRETT@sakilacustomer.org	285	1	2006-02-14 22:04:37	2006-02-15 04:57:20
276	1	BRANDY	GRAVES	BRANDY.GRAVES@sakilacustomer.org	281	1	2006-02-14 22:04:37	2006-02-15 04:57:20

200Lab Education

NA values in SQL

Van Anh - Data Mentor





Null Type

+ Definitions: NULL is a special value that is different from any valid value for any data type. However, the NULL value is a legal value in any data type. NULL is used to represent missing or inapplicable information.

+ 2 Types of NULL:

Situation	Description
missing	The field does have a value, but that value is unknown.
inapplicable	The field does not apply for this particular row.

What is NULL value?

+ **Definitions:** A field with a ***NULL value*** is a field with ***no value***.

+ If a field in a table is optional, it is possible to insert a new record or update a record without adding a value to this field. Then, the field will be saved with a ***NULL value***.



Note: A NULL value is different from a zero value or a field that contains spaces. A field with a NULL value is one that has been left blank during record creation!

How To Finding Null Value?

+ It is not possible to test for ***NULL*** values with comparison operators, such as **=, <, *or* <>.**

+ We will have to use the ***IS NULL*** and ***IS NOT NULL*** operators instead.



IS NULL Syntax:

```
SELECT column_names
FROM table_name
WHERE column_name IS NULL;
```

IS NOT NULL Syntax:

```
SELECT column_names
FROM table_name
WHERE column_name IS NOT NULL;
```

Some Examples

```
SELECT
    officeCode,
    city,
    phone,
    addressLine1,
    addressLine2,
    state,
    country,
    postalCode,
    territory
FROM ecommerce.offices
WHERE state IS NOT NULL;
```

	officeCode	city	phone	addressLine1	addressLine2	state
1	4	Paris	+33 14 723 4404	43 Rue Jouffroy D'abbans	[NULL]	[NULL]
2	6	Sydney	+61 2 9264 2451	5-11 Wentworth Avenue	Floor #2	[NULL]
3	7	London	+44 20 7877 2041	25 Old Broad Street	Level 7	[NULL]

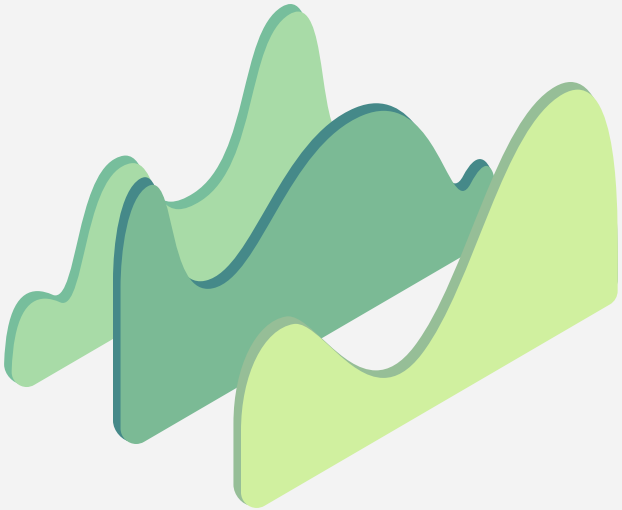
```
SELECT
    officeCode,
    city,
    phone,
    addressLine1,
    addressLine2,
    state,
    country,
    postalCode,
    territory
FROM ecommerce.offices
WHERE state IS NOT NULL;
```

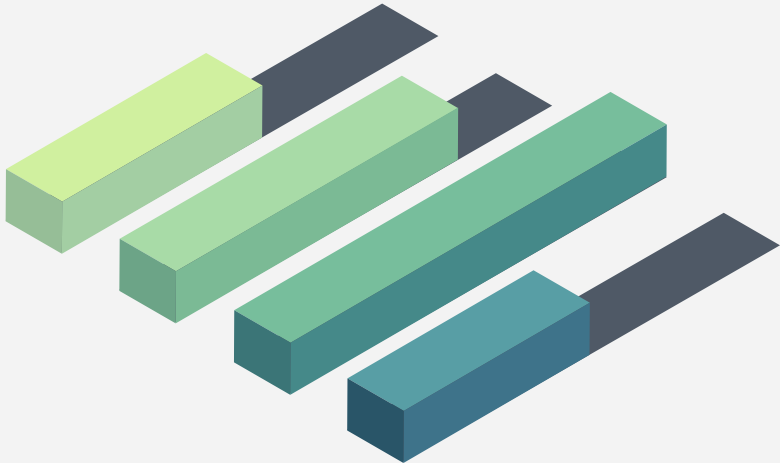
	officeCode	city	phone	addressLine1	addressLine2	state
1	1	San Francisco	+1 650 219 4782	100 Market Street	Suite 300	CA
2	2	Boston	+1 215 837 0825	1550 Court Place	Suite 102	MA
3	3	NYC	+1 212 555 3000	523 East 53rd Street	apt. 5A	NY
4	5	Tokyo	+81 33 224 5000	4-1 Kioicho	[NULL]	Chiyoda-Ku

200Lab Education

Conditional Expressions In SQL

Van Anh - Data Mentor





Definitions

+ Conditional expressions are used to define what logic is to be executed based on the status of some condition being satisfied.

+ Some conditional statements supported in SQL procedures:

- Case When
- Coalesce
- NULLIF
- IFNULL
- …



Case - When Statement

+ Definition: The ***CASE*** - ***WHEN*** statement goes through conditions and returns a value when the first condition is met (***like an if-then-else statement***). So, once a condition is true, it will stop reading and return the result. If no conditions are true, it returns the value in the ***ELSE*** clause.

+ If there is no ***ELSE*** part and no conditions are true, it returns ***NULL***.

+ Syntax:

```
CASE Expression
  WHEN expression1 Then Result1
  WHEN expression2 Then Result2
  WHEN expressionN Then ResultN
  ELSE Result
END;
```

Fig1: Simple CASE Statement

```
CASE
    WHEN condition1 THEN result1
    WHEN condition2 THEN result2
    WHEN conditionN THEN resultN
    ELSE result
END;
```

Fig2: CASE Statement With Comparison Operator

Simple Case - When Statement Example

```
SELECT
    title,
    release_year,
    rental_rate,
    `length`,
    rating,
    CASE rating
        WHEN 'G' THEN 'family'
        WHEN 'PG' THEN 'teens'
        WHEN 'PG-13' THEN 'teens'
        WHEN 'R' THEN 'adults'
        WHEN 'NC-17' THEN 'adults'
    END AS target_audience
FROM sakila.film
ORDER BY 1;
```

title	release_year	rental_rate	length	rating	target_audience
ACADEMY DINOSAUR	2006	0.99	86	PG	teens
ACE GOLDFINGER	2006	4.99	48	G	family
ADAPTATION HOLES	2006	2.99	50	NC-17	adults
AFFAIR PREJUDICE	2006	2.99	117	G	family
AFRICAN EGG	2006	2.99	130	G	family
AGENT TRUMAN	2006	2.99	169	PG	teens
AIRPLANE SIERRA	2006	4.99	62	PG-13	teens
AIRPORT POLLOCK	2006	4.99	54	R	adults
ALABAMA DEVIL	2006	2.99	114	PG-13	teens
ALADDIN CALENDAR	2006	4.99	63	NC-17	adults
ALAMO VIDEOTAPE	2006	0.99	126	G	family
ALASKA PHANTOM	2006	0.99	136	PG	teens
ALI FOREVER	2006	4.99	150	PG	teens
ALICE FANTASIA	2006	0.99	94	NC-17	adults
ALIEN CENTER	2006	2.99	46	NC-17	adults
ALLEY EVOLUTION	2006	2.99	180	NC-17	adults
ALONE TRIP	2006	0.99	82	R	adults
ALTER VICTORY	2006	0.99	57	PG-13	teens
AMADEUS HOLY	2006	0.99	113	PG	teens
AMELIE HELLFIGHTERS	2006	4.99	79	R	adults
AMERICAN CIRCUS	2006	4.99	129	R	adults
AMISTAD MIDSUMMER	2006	2.99	85	G	family
ANACONDA CONFESSIONS	2006	0.99	92	R	adults
ANALYZE HOOSIERS	2006	2.99	181	R	adults

Case - When Statement Example

```sql
SELECT
    payment_id,
    customer_id,
    amount,
    CASE
        WHEN amount < 3 THEN 'Low Price'
        WHEN amount >= 3 AND amount <= 7 THEN 'Medium Price'
        ELSE 'High Price'
    END AS price_type,
    payment_date
FROM sakila.payment
ORDER BY payment_id;
```

	payment_id	customer_id	amount	price_type	payment_date
1	1	1	2.99	Low Price	2005-05-25 11:30:37
2	2	1	0.99	Low Price	2005-05-28 10:35:23
3	3	1	5.99	Medium Price	2005-06-15 00:54:12
4	4	1	0.99	Low Price	2005-06-15 18:02:53
5	5	1	9.99	High Price	2005-06-15 21:08:46
6	6	1	4.99	Medium Price	2005-06-16 15:18:57
7	7	1	4.99	Medium Price	2005-06-18 08:41:48
8	8	1	0.99	Low Price	2005-06-18 13:33:59
9	9	1	3.99	Medium Price	2005-06-21 06:24:45
10	10	1	5.99	Medium Price	2005-07-08 03:17:05
11	11	1	5.99	Medium Price	2005-07-08 07:33:56
12	12	1	4.99	Medium Price	2005-07-09 13:24:07
13	13	1	4.99	Medium Price	2005-07-09 16:38:01
14	14	1	7.99	High Price	2005-07-11 10:13:46
15	15	1	2.99	Low Price	2005-07-27 11:31:22
16	16	1	4.99	Medium Price	2005-07-28 09:04:45
17	17	1	4.99	Medium Price	2005-07-28 16:18:23
18	18	1	0.99	Low Price	2005-07-28 17:33:39
19	19	1	0.99	Low Price	2005-07-28 19:20:07

Case - When Statement Example (Cont.)

```
SELECT
    address_id,
    address,
    CASE
        WHEN address2 IS NULL THEN 'unknown'
        ELSE address2
    END AS address2,
    district,
    city_id,
    postal_code,
    phone,
    location
FROM sakila.address;
```

	address_id	address	address2	district	city_id	postal_code	phone	location
1	1	47 MySakila Drive	unknown	Alberta	300			POINT (-112.818
2	2	28 MySQL Boulevard	unknown	QLD	576			POINT (153.140
3	3	23 Workhaven Lane	unknown	Alberta	300		14033335568	POINT (-112.818
4	4	1411 Lillydale Drive	unknown	QLD	576		6172235589	POINT (153.191
5	5	1913 Hanoi Way		Nagasaki	463	35200	28303384290	POINT (129.722
6	6	1121 Loja Avenue		California	449	17886	838635286649	POINT (-117.289
7	7	692 Joliet Street		Attika	38	83579	448477190408	POINT (23.7162
8	8	1566 Inegl Manor		Mandalay	349	53561	705814003527	POINT (95.3884
9	9	53 Idfu Parkway		Nantou	361	42399	10655648674	POINT (120.663
10	10	1795 Santiago de Compostela Way		Texas	295	18743	860452626434	POINT (-99.507
11	11	900 Santiago de Compostela Parkway		Central Serbia	280	93896	716571220373	POINT (0 0)
12	12	478 Joliet Way		Hamilton	200	77948	657282285970	POINT (175.283

Case - When Statement Example (Cont.)

```
SELECT
    film_id,
    title,
    rental_rate,
    rental_duration
FROM film
WHERE rental_duration = CASE rental_rate
                            WHEN 0.99 THEN 3
                            WHEN 2.99 THEN 4
                            WHEN 4.99 THEN 5
                            ELSE 6
                        END
ORDER BY title DESC;
```

film_id	title	rental_rate	rental_duration
995	YENTL IDAHO	4.99	5
991	WORST BANGER	2.99	4
990	WORLD LEATHERNECKS	0.99	3
978	WISDOM WORKER	0.99	3
966	WEDDING APOLLO	0.99	3
953	WAIT CIDER	0.99	3
939	VERTIGO NORTHWEST	2.99	4
932	VALLEY PACKER	0.99	3
930	VACATION BOONDOCK	2.99	4
929	USUAL UNTOUCHABLES	4.99	5
925	UNITED PILOT	0.99	3
920	UNBREAKABLE KARATE	0.99	3
913	TROOPERS METAL	0.99	3
910	TREATMENT JEKYLL	0.99	3
909	TREASURE COMMAND	0.99	3
903	TRAFFIC HOBBIT	4.99	5
901	TRACY CIDER	0.99	3
896	TOOTSIE PILOT	0.99	3
891	TIMBERLAND SKY	0.99	3
888	THIN SAGEBRUSH	4.99	5
887	THIEF PELICAN	4.99	5
884	TERMINATOR CLUB	4.99	5
881	TEMPLE ATTRACTION	4.99	5
873	SWEETHEARTS SUSPECTS	0.99	3
869	SUSPECTS QUILLS	2.99	4
867	SUPER WYOMING	4.99	5
858	SUBMARINE BED	4.99	5
856	STREETCAR INTENTIONS	4.99	5
851	STRAIGHT HOURS	0.99	3
848	STONE FIRE	0.99	3
842	STATE WASTELAND	2.99	4
838	STAGECOACH ARMAGEDD	4.99	5
837	STAGE WORLD	2.99	4
827	SPICE SORORITY	4.99	5
823	SOUTH WAIT	2.99	4
819	SONG HEDWIG	0.99	3
816	SNOWMAN ROLLERCOAS	0.99	3
815	SNATCHERS MONTEZUM	2.99	4
811	SMILE EARRING	2.99	4

Case - When Statement Example (Cont.)

```sql
SELECT
    *
FROM ecommerce.customers
ORDER BY (CASE city
              WHEN 'Las Vegas' THEN 1
              WHEN 'San Francisco' THEN 2
              WHEN 'Singapore' THEN 3
              ELSE 4
          END) ASC, city DESC;
```

erName	contactLastName	contactFirstName	phone	addressLine1	addressLine2	city	state	postalCode
Stores	King	Jean	7025551838	8489 Strong St.	[NULL]	Las Vegas	NV	83030
s Co.	Murphy	Julie	6505555787	5557 North Pendale Street	[NULL]	San Francisco	CA	94217
Gift Ideas Co.	Brown	Julie	6505551386	7734 Strong St.	[NULL]	San Francisco	CA	94217
veniers, Ltd.	Natividad	Eric	+65 221 7555	Bronz Sok.	Bronz Apt. 3/6 Tesvikiye	Singapore	[NULL]	079903
& Co	Victorino	Wendy	+65 224 1555	106 Linden Road Sandown	2nd Floor	Singapore	[NULL]	069045
ping Network, C	Walker	Brydey	+612 9411 1555	Suntec Tower Three	8 Temasek	Singapore	[NULL]	038988
lectables	Ibsen	Palle	86 21 3555	Smagsloget 45	[NULL]	Århus	[NULL]	8200
s	Frick	Steve	9145554562	3758 North Pendale Street	[NULL]	White Plains	NY	24067
sk Decorations,	McRoy	Sarah	04 499 9555	101 Lambton Quay	Level 11	Wellington	[NULL]	[NULL]
yszek Co	Piestrzeniewicz	Zbyszek	(26) 642-7555	ul. Filtrowa 68	[NULL]	Warszawa	[NULL]	01-012
és & Cie.	Tonini	Daniel	30.59.8555	67, avenue de l'Europe	[NULL]	Versailles	[NULL]	78000
ift Exchange Ne	Tamuri	Yoshi	(604) 555-3392	1900 Oak St.	[NULL]	Vancouver	BC	V3F 2K1
dian Collectable	Lincoln	Elizabeth	(604) 555-4555	23 Tsawassen Blvd.	[NULL]	Tsawassen	BC	T2F 8M4
ac	Roulet	Annette	61.77.6555	1 rue Alsace-Lorraine	[NULL]	Toulouse	[NULL]	31000
els & Co.	Accorti	Paolo	011-4988555	Via Monte Bianco 34	[NULL]	Torino	[NULL]	10100
llectable Excha	Müller	Rita	0711-555361	Adenauerallee 900	[NULL]	Stuttgart	[NULL]	70563
	Citeaux	Frédérique	88.60.1555	24, place Kléber	[NULL]	Strasbourg	[NULL]	67000
Imports	Bergulfsen	Jonas	07-98 9555	Erling Skakkes gate 78	[NULL]	Stavern	[NULL]	4110

COALESCE

+ **Definitions:** The MySQL ***COALESCE()*** function lets you return an alternative value if an expression is ***NULL***

+ **Syntax:**

```
COALESCE(val1, val2, ...., val_n)
```

+ **Example:**

```sql
SELECT
    address_id,
    address,
    COALESCE(address2, 'unknown') AS address2,
    district,
    city_id,
    postal_code,
    phone,
    location
FROM sakila.address;
```

	address_id	address	address2	district	city_id	postal_code
1	1	47 MySakila Drive	unknown	Alberta	300	
2	2	28 MySQL Boulevard	unknown	QLD	576	
3	3	23 Workhaven Lane	unknown	Alberta	300	
4	4	1411 Lillydale Drive	unknown	QLD	576	
5	5	1913 Hanoi Way		Nagasaki	463	35200
6	6	1121 Loja Avenue		California	449	17886
7	7	692 Joliet Street		Attika	38	83579
8	8	1566 Inegl Manor		Mandalay	349	53561
9	9	53 Idfu Parkway		Nantou	361	42399
10	10	1795 Santiago de Compostela Way		Texas	295	18743
11	11	900 Santiago de Compostela Parkway		Central Serbia	280	93896
12	12	478 Joliet Way		Hamilton	200	77948

COALESCE Example

```
SELECT
    COALESCE(NULL, '200lab', 'SQL', NULL)
```

COALESCE(NULL, '200lab', 'SQL', NULL)
200lab

```
SELECT
    COALESCE(NULL, NULL, NULL)
```

COALESCE(NULL, NULL, NULL)
[NULL]

```
SELECT
    COALESCE(NULL, NULL, 'SQL')
```

COALESCE(NULL, NULL, 'SQL')
SQL

NULLIF

+ **Definitions:** The ***NULLIF()*** function compares two expressions and returns ***NULL*** if they are equal. Otherwise, the first expression is returned.

+ **Syntax:**

```
NULLIF(expr1, expr2)
```

**Parameter:*

Parameter	Description
expr1, expr2	Required. The two expressions to be compared

NULLIF Example

```
SELECT
    NULLIF(25, 'Hello' );
```

	123 NULLIF(25, 'Hello')
1	25

```
SELECT
    NULLIF('Hello', 'Hello' );
```

	ABC NULLIF('Hello', 'Hello')
1	[NULL]

NULLIF Example (Cont.)

```sql
SELECT
    address_id,
    address,
    district,
    city_id,
    postal_code,
    NULLIF(phone, '') AS phone,
    location,
    last_update
FROM sakila.address
WHERE phone = '';
```

address_id	address	district	city_id	postal_code	phone	location	last_update
1	47 MySakila Drive	Alberta	300			POINT (-112.8185647 49.6999986)	2014-09-25 22:30:27
2	28 MySQL Boulevard	QLD	576			POINT (153.1408538 -27.6333361)	2014-09-25 22:30:09

address_id	address	district	city_id	postal_code	phone	location	last_update
1	47 MySakila Drive	Alberta	300		[NULL]	POINT (-112.8185647 49.6999986)	2014-09-25 22:30:27
2	28 MySQL Boulevard	QLD	576		[NULL]	POINT (153.1408538 -27.6333361)	2014-09-25 22:30:09

NULLIF Example (Cont.)

```sql
SELECT
    address_id,
    address,
    NULLIF(address2, '') AS address2,
    district,
    city_id,
    postal_code,
    phone,
    location,
    last_update
FROM sakila.address;
```

address_id	address	address2	district	city_id	postal_code	phone
1	47 MySakila Drive	[NULL]	Alberta	300		
2	28 MySQL Boulevard	[NULL]	QLD	576		
3	23 Workhaven Lane	[NULL]	Alberta	300		14033335568
4	1411 Lillydale Drive	[NULL]	QLD	576		6172235589
5	1913 Hanoi Way		Nagasaki	463	35200	28303384290
6	1121 Loja Avenue		California	449	17886	838635286649
7	692 Joliet Street		Attika	38	83579	448477190408
8	1566 Inegl Manor		Mandalay	349	53561	705814003527
9	53 Idfu Parkway		Nantou	361	42399	10655648674
10	1795 Santiago de Compostela Way		Texas	295	18743	860452626434
11	900 Santiago de Compostela Parkway		Central Serbia	280	93896	716571220373
12	478 Joliet Way		Hamilton	200	77948	657282285970
13	613 Korolev Drive		Masqat	329	45844	380657522649
14	1531 Sal Drive		Esfahan	162	53628	648856936185
15	1542 Tarlac Parkway		Kanagawa	440	1027	635297277345
16	808 Bhopal Manor		Haryana	582	10672	465887807014

address_id	address	address2	district	city_id	postal_code	phone
1	47 MySakila Drive	[NULL]	Alberta	300		
2	28 MySQL Boulevard	[NULL]	QLD	576		
3	23 Workhaven Lane	[NULL]	Alberta	300		14033335568
4	1411 Lillydale Drive	[NULL]	QLD	576		6172235589
5	1913 Hanoi Way	[NULL]	Nagasaki	463	35200	28303384290
6	1121 Loja Avenue	[NULL]	California	449	17886	838635286649
7	692 Joliet Street	[NULL]	Attika	38	83579	448477190408
8	1566 Inegl Manor	[NULL]	Mandalay	349	53561	705814003527
9	53 Idfu Parkway	[NULL]	Nantou	361	42399	10655648674
10	1795 Santiago de Compostela Way	[NULL]	Texas	295	18743	860452626434
11	900 Santiago de Compostela Parkway	[NULL]	Central Serbia	280	93896	716571220373
12	478 Joliet Way	[NULL]	Hamilton	200	77948	657282285970
13	613 Korolev Drive	[NULL]	Masqat	329	45844	380657522649
14	1531 Sal Drive	[NULL]	Esfahan	162	53628	648856936185
15	1542 Tarlac Parkway	[NULL]	Kanagawa	440	1027	635297277345
16	808 Bhopal Manor	[NULL]	Haryana	582	10672	465887807014

IFNULL

+ **Definitions:** The ***IFNULL()*** function returns a specified value if the expression is ***NULL***.

+ If the expression is ***NOT NULL***, this function returns the expression.

+ **Syntax:**

```
IFNULL(expression, alt_value)
```

**Parameter:*

Parameter	Description
expression	Required. The expression to test whether is NULL
alt_value	Required. The value to return if *expression* is NULL

IFNULL Examples

```sql
SELECT
    address_id,
    address,
    IFNULL(address2, 'unknown') AS address2,
    district,
    city_id,
    postal_code,
    phone,
    location
FROM sakila.address;
```

	address_id	address	address2	district	city_id	postal_code
1	1	47 MySakila Drive	unknown	Alberta	300	
2	2	28 MySQL Boulevard	unknown	QLD	576	
3	3	23 Workhaven Lane	unknown	Alberta	300	
4	4	1411 Lillydale Drive	unknown	QLD	576	
5	5	1913 Hanoi Way		Nagasaki	463	35200
6	6	1121 Loja Avenue		California	449	17886
7	7	692 Joliet Street		Attika	38	83579
8	8	1566 Inegl Manor		Mandalay	349	53561
9	9	53 Idfu Parkway		Nantou	361	42399
10	10	1795 Santiago de Compostela Way		Texas	295	18743
11	11	900 Santiago de Compostela Parkway		Central Serbia	280	93896
12	12	478 Joliet Way		Hamilton	200	77948

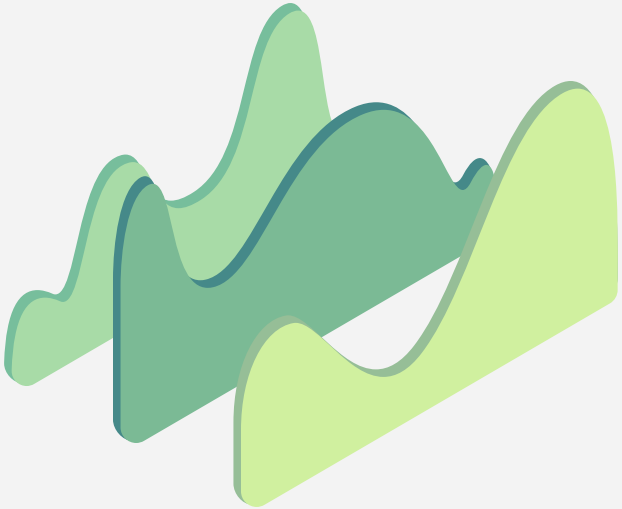
```sql
SELECT
    customerNumber,
    customerName,
    IFNULL(salesRepEmployeeNumber, 0) AS salesRepEmployeeNumber,
    creditLimit
FROM ecommerce.customers
WHERE customerNumber BETWEEN 120 AND 130
ORDER BY 1;
```

	customerNumber	customerName	salesRepEmployeeNumber
1	121	Baane Mini Imports	1,504
2	124	Mini Gifts Distributors Ltd.	1,165
	125	Havel & Zbyszek Co	0
	128	Blauer See Auto, Co.	1,504
	129	Mini Wheels Co.	1,165

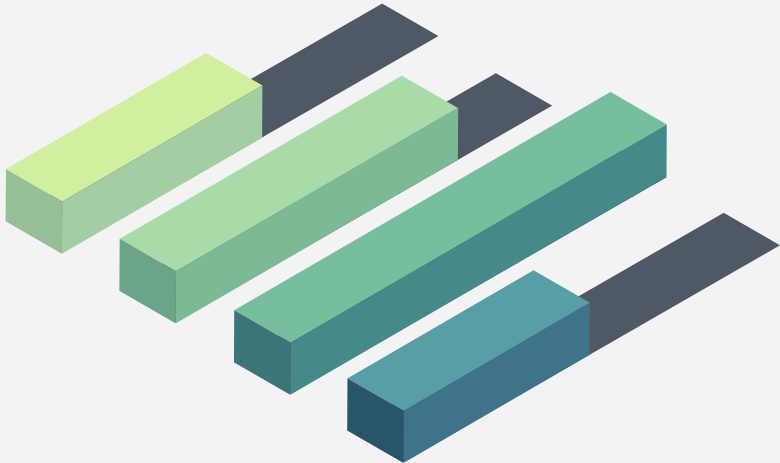
	customerNumber	customerName	salesRepEmployeeNumber
	121	Baane Mini Imports	1,504
2	124	Mini Gifts Distributors Ltd.	1,165
3	125	Havel & Zbyszek Co	[NULL]
4	128	Blauer See Auto, Co.	1,504
5	129	Mini Wheels Co.	1,165

200Lab Education

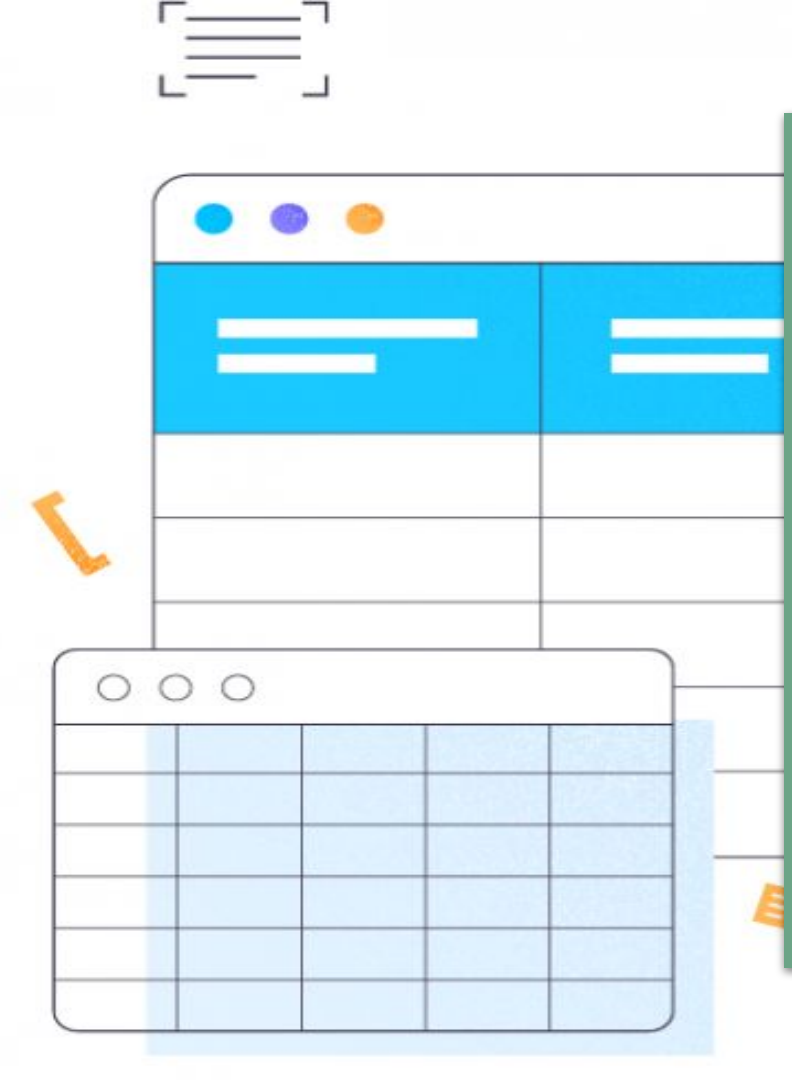
Keyword Takeaway



Van Anh - Data Mentor



200Lab Education



Data Type In SQL

Data Type In SQL

1. String Data Types

Data type	Description
CHAR(size)	A FIXED length string (can contain letters, numbers, and special characters). The *size* parameter specifies the column length in characters - can be from 0 to 255. Default is 1
VARCHAR(size)	A VARIABLE length string (can contain letters, numbers, and special characters). The *size* parameter specifies the maximum column length in characters - can be from 0 to 65535
BINARY(size)	Equal to CHAR(), but stores binary byte strings. The *size* parameter specifies the column length in bytes. Default is 1
VARBINARY(size)	Equal to VARCHAR(), but stores binary byte strings. The *size* parameter specifies the maximum column length in bytes.
TINYBLOB	For BLOBs (Binary Large OBjects). Max length: 255 bytes
TINYTEXT	Holds a string with a maximum length of 255 characters
TEXT(size)	Holds a string with a maximum length of 65,535 bytes
BLOB(size)	For BLOBs (Binary Large OBjects). Holds up to 65,535 bytes of data
MEDIUMTEXT	Holds a string with a maximum length of 16,777,215 characters
MEDIUMBLOB	For BLOBs (Binary Large OBjects). Holds up to 16,777,215 bytes of data
LONGTEXT	Holds a string with a maximum length of 4,294,967,295 characters
LONGBLOB	For BLOBs (Binary Large OBjects). Holds up to 4,294,967,295 bytes of data
ENUM(val1, val2, val3, ...)	A string object that can have only one value, chosen from a list of possible values. You can list up to 65535 values in an ENUM list. If a value is inserted that is not in the list, a blank value will be inserted. The values are sorted in the order you enter them
SET(val1, val2, val3, ...)	A string object that can have 0 or more values, chosen from a list of possible values. You can list up to 64 values in a SET list

Data Type In SQL (Cont)

2. Numeric Data Types

Data type	Description
BIT(*size*)	A bit-value type. The number of bits per value is specified in *size*. The *size* parameter can hold a value from 1 to 64. The default value for *size* is 1.
TINYINT(*size*)	A very small integer. Signed range is from -128 to 127. Unsigned range is from 0 to 255. The *size* parameter specifies the maximum display width (which is 255)
BOOL	Zero is considered as false, nonzero values are considered as true.
BOOLEAN	Equal to BOOL
SMALLINT(*size*)	A small integer. Signed range is from -32768 to 32767. Unsigned range is from 0 to 65535. The *size* parameter specifies the maximum display width (which is 255)
MEDIUMINT(*size*)	A medium integer. Signed range is from -8388608 to 8388607. Unsigned range is from 0 to 16777215. The *size* parameter specifies the maximum display width (which is 255)
INT(*size*)	A medium integer. Signed range is from -2147483648 to 2147483647. Unsigned range is from 0 to 4294967295. The *size* parameter specifies the maximum display width (which is 255)
INTEGER(*size*)	Equal to INT(size)
BIGINT(*size*)	A large integer. Signed range is from -9223372036854775808 to 9223372036854775807. Unsigned range is from 0 to 18446744073709551615. The *size* parameter specifies the maximum display width (which is 255)
FLOAT(*size*, *d*)	A floating point number. The total number of digits is specified in *size*. The number of digits after the decimal point is specified in the *d* parameter. This syntax is deprecated in MySQL 8.0.17, and it will be removed in future MySQL versions
FLOAT(*p*)	A floating point number. MySQL uses the *p* value to determine whether to use FLOAT or DOUBLE for the resulting data type. If *p* is from 0 to 24, the data type becomes FLOAT(). If *p* is from 25 to 53, the data type becomes DOUBLE()
DOUBLE(*size*, *d*)	A normal-size floating point number. The total number of digits is specified in *size*. The number of digits after the decimal point is specified in the *d* parameter
DOUBLE PRECISION(*size*, *d*)	
DECIMAL(*size*, *d*)	An exact fixed-point number. The total number of digits is specified in *size*. The number of digits after the decimal point is specified in the *d* parameter. The maximum number for *size* is 65. The maximum number for *d* is 30. The default value for *size* is 10. The default value for *d* is 0.
DEC(*size*, *d*)	Equal to DECIMAL(size,d)

Data Type In SQL (Cont)

3. Date and Time Data Types

Data type	Description
DATE	A date. Format: YYYY-MM-DD. The supported range is from '1000-01-01' to '9999-12-31'
DATETIME(*fsp*)	A date and time combination. Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'. Adding DEFAULT and ON UPDATE in the column definition to get automatic initialization and updating to the current date and time
TIMESTAMP(*fsp*)	A timestamp. TIMESTAMP values are stored as the number of seconds since the Unix epoch ('1970-01-01 00:00:00' UTC). Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1970-01-01 00:00:01' UTC to '2038-01-09 03:14:07' UTC. Automatic initialization and updating to the current date and time can be specified using DEFAULT CURRENT_TIMESTAMP and ON UPDATE CURRENT_TIMESTAMP in the column definition
TIME(*fsp*)	A time. Format: hh:mm:ss. The supported range is from '-838:59:59' to '838:59:59'
YEAR	A year in four-digit format. Values allowed in four-digit format: 1901 to 2155, and 0000. MySQL 8.0 does not support year in two-digit format.

200Lab Education



Format Query SQL

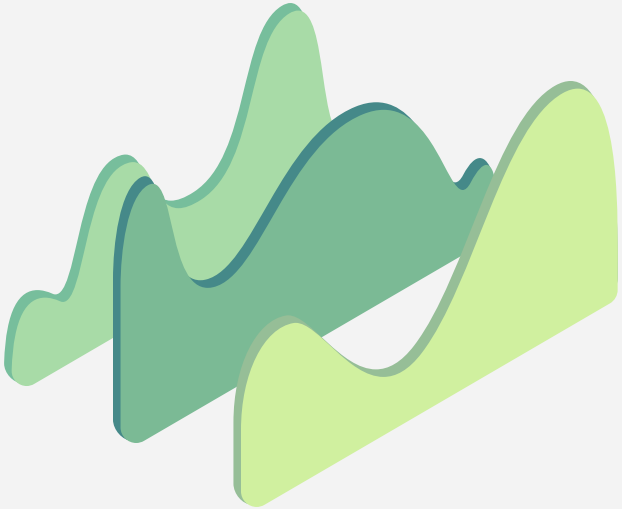
Format Query In MySQL (Cont)



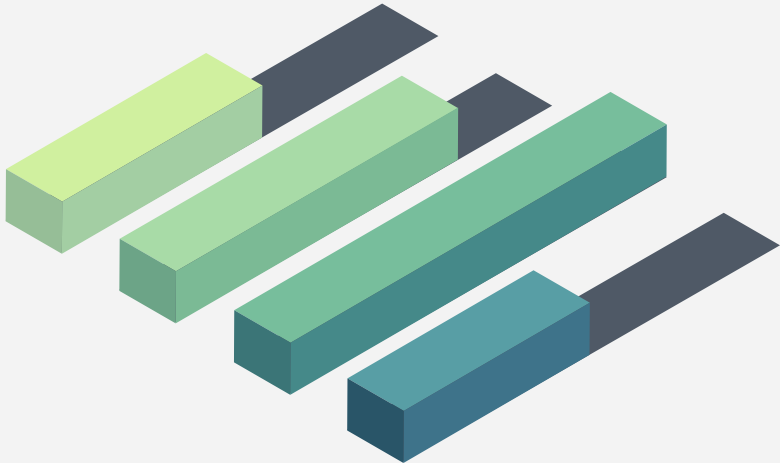

```
SELECT
    [DISTINCT ]
    columns | *
FROM table_name
[WHERE clause ]
[GROUP BY clause ]
[HAVING BY clause ]
[ORDER BY clause ]
```

200Lab Education

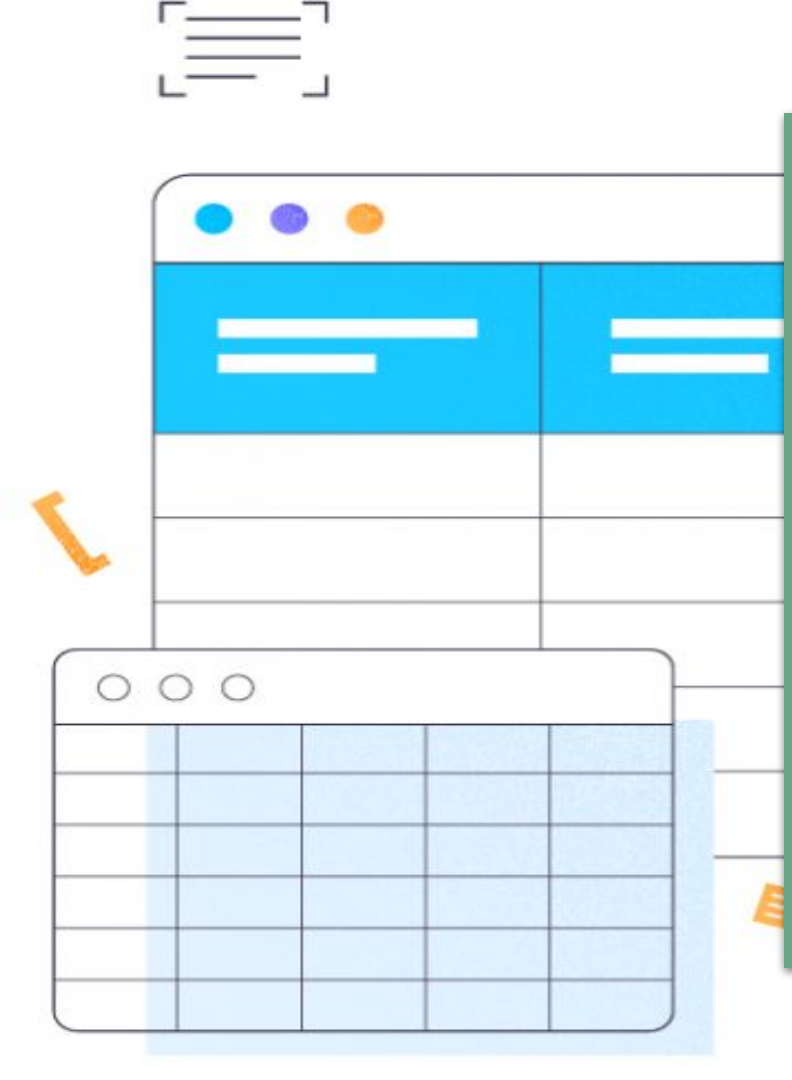
Keyword Takeaway



Van Anh - Data Mentor



200Lab Education



Data Type In SQL

Data Type In SQL

1. String Data Types

Data type	Description
CHAR(size)	A FIXED length string (can contain letters, numbers, and special characters). The *size* parameter specifies the column length in characters - can be from 0 to 255. Default is 1
VARCHAR(size)	A VARIABLE length string (can contain letters, numbers, and special characters). The *size* parameter specifies the maximum column length in characters - can be from 0 to 65535
BINARY(size)	Equal to CHAR(), but stores binary byte strings. The *size* parameter specifies the column length in bytes. Default is 1
VARBINARY(size)	Equal to VARCHAR(), but stores binary byte strings. The *size* parameter specifies the maximum column length in bytes.
TINYBLOB	For BLOBs (Binary Large OBjects). Max length: 255 bytes
TINYTEXT	Holds a string with a maximum length of 255 characters
TEXT(size)	Holds a string with a maximum length of 65,535 bytes
BLOB(size)	For BLOBs (Binary Large OBjects). Holds up to 65,535 bytes of data
MEDIUMTEXT	Holds a string with a maximum length of 16,777,215 characters
MEDIUMBLOB	For BLOBs (Binary Large OBjects). Holds up to 16,777,215 bytes of data
LONGTEXT	Holds a string with a maximum length of 4,294,967,295 characters
LONGBLOB	For BLOBs (Binary Large OBjects). Holds up to 4,294,967,295 bytes of data
ENUM(val1, val2, val3, ...)	A string object that can have only one value, chosen from a list of possible values. You can list up to 65535 values in an ENUM list. If a value is inserted that is not in the list, a blank value will be inserted. The values are sorted in the order you enter them
SET(val1, val2, val3, ...)	A string object that can have 0 or more values, chosen from a list of possible values. You can list up to 64 values in a SET list

Data Type In SQL (Cont)

2. Numeric Data Types

Data type	Description
BIT(*size*)	A bit-value type. The number of bits per value is specified in *size*. The *size* parameter can hold a value from 1 to 64. The default value for *size* is 1.
TINYINT(*size*)	A very small integer. Signed range is from -128 to 127. Unsigned range is from 0 to 255. The *size* parameter specifies the maximum display width (which is 255)
BOOL	Zero is considered as false, nonzero values are considered as true.
BOOLEAN	Equal to BOOL
SMALLINT(*size*)	A small integer. Signed range is from -32768 to 32767. Unsigned range is from 0 to 65535. The *size* parameter specifies the maximum display width (which is 255)
MEDIUMINT(*size*)	A medium integer. Signed range is from -8388608 to 8388607. Unsigned range is from 0 to 16777215. The *size* parameter specifies the maximum display width (which is 255)
INT(*size*)	A medium integer. Signed range is from -2147483648 to 2147483647. Unsigned range is from 0 to 4294967295. The *size* parameter specifies the maximum display width (which is 255)
INTEGER(*size*)	Equal to INT(size)
BIGINT(*size*)	A large integer. Signed range is from -9223372036854775808 to 9223372036854775807. Unsigned range is from 0 to 18446744073709551615. The *size* parameter specifies the maximum display width (which is 255)
FLOAT(*size*, *d*)	A floating point number. The total number of digits is specified in *size*. The number of digits after the decimal point is specified in the *d* parameter. This syntax is deprecated in MySQL 8.0.17, and it will be removed in future MySQL versions
FLOAT(*p*)	A floating point number. MySQL uses the *p* value to determine whether to use FLOAT or DOUBLE for the resulting data type. If *p* is from 0 to 24, the data type becomes FLOAT(). If *p* is from 25 to 53, the data type becomes DOUBLE()
DOUBLE(*size*, *d*)	A normal-size floating point number. The total number of digits is specified in *size*. The number of digits after the decimal point is specified in the *d* parameter
DOUBLE PRECISION(*size*, *d*)	
DECIMAL(*size*, *d*)	An exact fixed-point number. The total number of digits is specified in *size*. The number of digits after the decimal point is specified in the *d* parameter. The maximum number for *size* is 65. The maximum number for *d* is 30. The default value for *size* is 10. The default value for *d* is 0.
DEC(*size*, *d*)	Equal to DECIMAL(size,d)

Data Type In SQL (Cont)

3. Date and Time Data Types

Data type	Description
DATE	A date. Format: YYYY-MM-DD. The supported range is from '1000-01-01' to '9999-12-31'
DATETIME(*fsp*)	A date and time combination. Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'. Adding DEFAULT and ON UPDATE in the column definition to get automatic initialization and updating to the current date and time
TIMESTAMP(*fsp*)	A timestamp. TIMESTAMP values are stored as the number of seconds since the Unix epoch ('1970-01-01 00:00:00' UTC). Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1970-01-01 00:00:01' UTC to '2038-01-09 03:14:07' UTC. Automatic initialization and updating to the current date and time can be specified using DEFAULT CURRENT_TIMESTAMP and ON UPDATE CURRENT_TIMESTAMP in the column definition
TIME(*fsp*)	A time. Format: hh:mm:ss. The supported range is from '-838:59:59' to '838:59:59'
YEAR	A year in four-digit format. Values allowed in four-digit format: 1901 to 2155, and 0000. MySQL 8.0 does not support year in two-digit format.

200Lab Education



Format Query SQL

Format Query In MySQL (Cont)




```
SELECT
    [DISTINCT ]
    columns | *
FROM table_name
[WHERE clause ]
[GROUP BY clause ]
[HAVING BY clause ]
[ORDER BY clause ]
```